Solution Architecture

Date	28 June 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	5 Marks

Solution Architecture Diagram:

The Solution Architecture Diagram provides a structured, multi-layer visual roadmap of the RISING WATERS system, mapping the journey of meteorological data across specialized user, presentation, logic, machine learning, data processing, and cloud deployment pipelines. It illustrates exactly how a user web interface connects through a Flask backend to invoke a highly accurate XGBoost prediction engine.

Presentation / Client Layer

Web Application User Interface built with **HTML, CSS, JavaScript** containing a **Home Page (Dashboard), Input Form (Weather Parameters), Prediction Result page**, and **Prediction History** panel.

↓↑

API Gateway / Load Balancer

Flask Route Handling (URL Mapping) acting as the core gateway matching client actions directly to backend server processes.

↓↑

Auth Service	Core Logic Service	External API Integrations
Flask-Login session management system executing secure validation checks of User Login Credentials against the authenticated Users Store.	**Flask Backend Controller** executing Form Validation, Data Preprocessing (Scaling), loading the trained machine learning model (joblib), and generating predictions.	Real-time meteorological data feeds connected to weather forecasting service APIs to pull localized live rain and visibility parameters.

↓↑

Data / Storage Layer

Tabular **Flood Prediction Dataset (CSV format from Kaggle)** , tracking historical features like annual rainfall, cloud visibility, and seasonal features , along with **Prediction Results & History Store**.

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Provides an intuitive, multi-page frontend dashboard where system users (residents, meteorologists, relief coordinators) can view dashboards, input localized weather parameters, and receive instant alerts.	HTML, CSS, JavaScript, Flask UI templates
API Gateway / Core Logic	Handles incoming traffic routes, maps client URLs, validates input parameter web forms, applies data preprocessing steps, and feeds clean inputs directly into the saved model.	Flask Web Framework, Python
Microservices / ML Layer	Evaluates critical meteorological inputs (annual rainfall, cloud visibility, and seasonal patterns) against the saved classification algorithm to produce a real-time risk status.	XGBoost Classifier, Scikit-learn, Joblib/Pickle, Model (model.pkl), Scaler (scaler.pkl)
Database / Storage Layer	Holds open-source historical climate datasets for preprocessing and baseline metric training, while simultaneously logging transaction requests and historical prediction values.	CSV tabular files, Local Storage System / Cloud Storage instances